

33 LinkedList Add two numbers represented by linked lists.

```
class Solution {
private:
    Node* rev(Node* head) {
        Node* prev = NULL;
        while (head) {
            Node* nxt = head->next;
            head->next = prev;
            prev = head;
            head = nxt;
        }
        return prev;
    }
public:
    Node* addTwoLists(Node* h1, Node* h2) {
        h1 = rev(h1);
        h2 = rev(h2);
        int carry = 0;
        Node *head = NULL;
        while (h1 || h2 || carry) {
            int sum = carry;
            if (h1) sum += h1->data, h1 = h1->next;
            if (h2) sum += h2->data, h2 = h2->next;

            Node* n = new Node(sum % 10);
            n->next = head;    // build in reverse directly
            head = n;
            carry = sum / 10;
        }
        // Trim leading zeros (but leave single zero if result is 0)
        while (head && head->data == 0 && head->next) head = head->next;
        return head;
    }
};
```

Output Window

Compilation Results Custom Input Y.O.G.I. (AI Bot)

Problem Solved Successfully [Suggest Feedback](#)

Test Cases Passed
1111 / 1111

Attempts: Correct / Total
1 / 3

Accuracy: 33%

Points Scored **4 / 4**

Time Taken **0.18**

Your Total Score: 120 [↑](#)

Solve Next

[Intersection in Y Shaped Lists](#) [Merge two sorted linked lists](#) [Flattening a Linked List](#)

Stay Ahead With:

Build 31 Projects in 31 Days

```
1 class Solution {
2     private:
3         Node* rev(Node* head) {
4             Node* prev = NULL;
5             while (head) {
6                 Node* nxt = head->next;
7                 head->next = prev;
8                 prev = head;
9                 head = nxt;
10            }
11            return prev;
12        }
13    public:
14        Node* addTwoLists(Node* h1, Node* h2) {
15            h1 = rev(h1);
16            h2 = rev(h2);
17            int carry = 0;
18            Node *head = NULL;
19            while (h1 || h2 || carry) {
20                int sum = carry;
21                if (h1) sum += h1->data, h1 = h1->next;
22                if (h2) sum += h2->data, h2 = h2->next;
23
24                Node* n = new Node(sum % 10);
25                n->next = head;    // build in reverse directly
26                head = n;
27                carry = sum / 10;
28            }
29            // Trim leading zeros (but leave single zero if result is 0)
30            while (head && head->data == 0 && head->next) head = head->next;
31            return head;
32        }
33    };
34 }
```